Project 2: Mean-Shift Tracker

Katarina Brešar

I. Introduction

In this assignment we implement the mean shift algorithm for seeking the modes of a distribution. We then apply this algorithm to the problem of tracking an object in a video sequence. We test it on different sequences and analyze the results.

II. Experiments

A. Mean-shift mode seeking

First, we implemented the mean shift algorithm and tested it on two synthetic response maps. We first tested how the starting point affect the convergence and the final position of the algorithm. In figure 1a we can see how the choice in starting position affects the convergence. All three starting positions convergence towards a mode, but when the starting point is closer to the local maximum it will converge towards it instead of the proper mode. In figure 1b we can see how the choice of kernel affects the convergence. The synthetic response map has two distinct peaks (both with same value). The starting points are at the same distance from the two peaks. The mean shift with the gaussian kernel converges towards the top left peak and while it takes a smoother path it also takes more iterations to converge, while on the other hand using the epanechnikov kernel it covnerges towards the bottom right peak and it takes a more direct path and less iterations to converge. In figure 1c it's clear that if the size of the kernel is too small it will not converge towards the mode (not even the local maximum). In this case setting the kernel size to 7 was still too small and started to converge towards the local maximum, but while setting the kernel size to 10 meant that the mean shift 'could see' far enough and therefore found its way towards the mode.

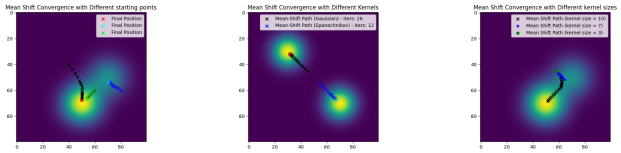
To speed up convergence we can add a relaxing factor to the mean shift update. So instead of just updating the new position of the mean we add a factor $\lambda > 1$ so that: $x_{k+1} = x_k + \lambda m(x_k)$. But this can lead to overshooting if the factor is too high. To fix that we compare the relaxed position to the non-relaxed position and if it's larger (so closer to the mode) we use that, if not we use the non-relaxed position and reduce the relaxation by 20%. Note that using $\lambda = 1$ is the same as not using relaxation at all. In table I we see that the higher the factor is the faster the convergence, but if it's too high (despite the corrections) it can lead to overshooting and therefore a worse final position.

Relax Factor	Iterations	Distance from True Mode
1.0	21	4.58
1.5	15	5.84
3.0	10	3.34
5.0	9	0.93
6.5	9	1.79
8.0	6	0.83

Table I: Speed of convergence and distance from true mode with different relax factors.

B. Mean-shift tracker

The mean-shift tracker is a simple tracking algorithm that uses the mean shift algorithm to find the mode of the distribution of the target in the current frame. We tested the tracker on five sequences from the VOT14 dataset. The tracking performance is shown in table II. In it we can see that the tracker performed well on all sequences losing the target only twice in the fernando and bolt sequences and once in the drunk sequence.



(a) Starting points. (b) Kernels. (c) Kernel sizes.

Figure 1: Mean shift convergence with change in parameters.

Sequence	Number of Failures	Frames per Second
drunk	1	587.5
fernando	2	189.3
bolt	2	450.5
car	0	562.4
polarbear	0	927.4

Table II: Tracking performance on five sequences.

In the bolt sequence the tracker lost the target twice. The first time was when the target was moving closer to the camera (and therefore getting bigger) while also running faster so the failure likely happened because the jump from frame to frame was too big for the tracker to follow (the runner moved too fast). At the same time another runner entered the frame and the tracker likely got confused and started following the wrong target. In the fernando sequence the target was a cat fighting with another cat. The tracker failed when the second cat occluded the target one and the tracker followed the wrong cat. In the drunk sequence the target was a car being driven by a drunk driver recorded by a security camera (stationary camera). The tracker failed when the car moved mostly out of the frame and only a small part of it was visible. When it moved back into the frame the tracker first failed to find it. So the histogram (that was tracking the back of the camera before it went out of frame) changed a lot when it moved back and the tracker failed to find the target. For the other two sequences the tracker performed well and didn't lose the target. Interestingly the tracker didn't fail on the car sequence. But the car was moving closer and closer to the camera and getting bigger, so the tracker wasn't really following the whole car but just a part of it (since the frame stayed the same size the whole time). But it wasn't a failure since the tracker was still following the right object. The tracker also performed well on the polarbear sequence, which makes sense since the target was moving slowly and the background stayed mostly the same. There was nothing to distrupt the tracker and the target was always visible.

To improve the robustness of the tracker we could add a scale estimation to it so that it can adapt to changes in the size of the target. To make sure it doesn't get confused by other objects that get into the background of the frame we could search for the features of the target by focusing on what discriminates it from the background instead of just looking at the histogram of the target.

III. Conclusion